

GridPulse Milestone 3 — Video Script

Team 9 · 3 presenters · about 10 minutes · every command is also in **GridPulse_M3_Video_Commands.txt** for copy-paste

PERSON 1

Balavardhan

Intro, architecture, laptop run, close · drives the screen

PERSON 2

Name: _____

Dockerfile & secrets, Artifact Registry, what's easier

PERSON 3

Name: _____

VM pull & run, budget

Who drives the screen: record a Zoom or Teams call with all three cameras on. Balavardhan shares his laptop screen for the whole video, since it has Docker, the GCP login and the VM SSH window. Persons 2 and 3 talk with their cameras on while he clicks; agree on a cue such as "next, please". Each speaker's face must be visible during their part.

Run of show

Time	Segment	Speaker	On screen	Rubric
0:00–0:45	Intro	Person 1	Cameras / deck title	—
0:45–1:45	Container architecture	Person 1	Slide 1	Slides
1:45–3:45	docker compose up on the laptop	Person 1	Terminal, localhost	Runs locally (10)
3:45–4:45	Dockerfile, .dockerignore, no secrets	Person 2	VS Code, evidence PDF p. 6	Dockerfile & secrets (8)
4:45–5:45	Artifact Registry: v1 tag and digest	Person 2	Registry pages, terminal	Version tag (7)
5:45–8:15	Pull and run on the VM, same URL	Person 3	VM SSH, browser	Same image on VM (10)
8:15–9:00	Budget alerts and current spend	Person 3	Billing pages	Budget (5)
9:00–9:45	What's easier than Milestone 2	Person 2	Slide 2	Required
9:45–10:00	Close	Person 1 (all on camera)	Cameras	—

Before you hit record (off camera)

- ☐ Run **section A** of the commands file: `laptop docker compose down` and `./deploy/allow_my_ip.sh`; on the VM, `docker compose down` and the `docker image rm ...` line.
- ☐ Open the browser tabs listed in section A: both Artifact Registry pages, VM instances, Budgets & alerts, Billing overview.

- ☐ Open the deck (slide 1), `backend/Dockerfile` and `.dockerignore` in VS Code, and the evidence PDF on page 6.
- ☐ Terminal font bigger (⌘ +), browser zoom 125%, notifications off, a small PDF on the Desktop.
- ☐ Never show `.env` (database password) or billing card details on screen.

The script

0:00–0:45 · Intro

PERSON 1

SHOW

All three cameras, or the deck's title.

SAY

"Hi everyone, we're Team 9 and this is GridPulse, our smart grid load forecasting and peak-risk alerting platform. I'm Balavardhan, and with me are [Person 2] and [Person 3]. In Milestone 2 we did a manual lift and shift onto a Google Cloud VM: we installed Postgres, Python and Node by hand and started three processes ourselves. In Milestone 3 we packaged the whole app into containers, pushed a versioned image to Artifact Registry, and ran that exact same image on the VM. So the container on our laptop and the container on the VM are byte-for-byte the same artifact, and we'll prove it with the image digest."

0:45–1:45 · Container architecture

PERSON 1

SHOW

Slide 1. Point at each box as you mention it.

SAY

"Here's our container architecture. One docker-compose file starts three containers. The frontend is nginx serving our React build on port 80, and it also forwards every /api call to the backend. The backend is FastAPI with our trained XGBoost model baked into the image, on port 8000. And the database is the official Postgres 16 image, with no port open to the outside at all. We use two networks: web-net connects nginx to the backend, and data-net connects the backend to Postgres, so the frontend has no route to the database. Two named volumes, pgdata and uploads, keep our data when containers restart or get upgraded. And at the bottom is the flow: build once on the laptop, push to Artifact Registry, pull and run on the VM."

1:45–3:45 · docker compose up on the laptop

PERSON 1

DO — COMMANDS FILE B1

```
cd ~/Downloads/gridpulse
docker compose up -d --wait
docker compose ps
```

1. Point at the three **(healthy)** containers and the db row showing only 5432/tcp .
2. Browser **http://localhost**: Dashboard (10 regions) → Regional forecast → pick a region → **Recompute forecast** → Documents → upload the PDF → Alerts → Async jobs → **Simulate spike**.
3. **http://localhost:8000/docs** for a few seconds.

SAY

"I'm on my laptop. I run docker compose up, and compose starts the database first, waits until it's healthy, then the backend, then the frontend. All three containers are healthy, and notice the database shows 5432/tcp with no 0.0.0.0, so it's only reachable inside Docker. Here's the dashboard at localhost with all 10 regions. When I recompute a forecast, it comes from the real XGBoost model inside the container. I'll upload an infrastructure PDF, which is saved in the uploads volume, and on Async jobs I'll simulate a spike: the background worker processes the jobs while the app stays responsive. Everything here runs in containers. The only thing installed on this Mac is Docker."

Hand-off: "Now [Person 2] will show how the image is built."

3:45–4:45 · Dockerfile, .dockerignore, no secrets

PERSON 2

SHOW

1. backend/Dockerfile in VS Code: the two FROM lines (two stages), the xgboost-cpu line, USER app .
2. .dockerignore: .env , .git , node_modules , venv , uploads .
3. Evidence PDF, page 6: the secrets check with every line **PASS**. No command is needed; the script takes 2 minutes, so show the saved result.

SAY

"Quickly on how the image is built. The backend Dockerfile has two stages: a builder stage installs the Python packages, and only the finished virtual environment is copied into the final image. We swapped xgboost for xgboost-cpu, which drops GPU libraries we never use, and that took the compressed image from 821 megabytes down to 192, which is 77 percent smaller. The app also runs as a non-root user. Our .dockerignore keeps out the .env file with the database password, the .git folder, the Mac's node_modules and venv, and uploads, so the whole build context is just 1.2 megabytes. And the password is never in the image: Docker Compose injects it at run time from .env. We verified that with a script that checks the image settings, the build history, searches for .env files and scans every layer for the real password. Every check passed."

4:45–5:45 · Artifact Registry: v1 tag and digest

PERSON 2

SHOW

1. Artifact Registry → gridpulse → **gridpulse-backend** → version page: **Tags: latest, v1** and the **Digest**.
2. The same page for **gridpulse-frontend**.
3. Laptop terminal — commands file B2:

```
docker image inspect --format '{{index .RepoDigests 0}}' us-central1-docker.pkg.dev/gridpulse-509202/gridpulse/gridpulse-backend:v1
```

SAY

"Now Artifact Registry. This is our gridpulse repository in us-central1, the same region as our VM. Here's the backend image with two tags: latest, and our explicit version tag, v1. And this is its digest, sha256 starting with 6-8-7-b-3-2-e-2. The digest is a fingerprint of the exact bytes, so if even one byte changed, it would be different. The frontend is here too, with a digest starting 4-0-3-8-1-6. And on our laptop, docker image inspect shows the same backend digest, 6-8-7-b-3-2-e-2. Remember that number, because we'll check it on the VM."

Hand-off: "[Person 3] will now run this exact image on our VM."

5:45–8:15 · Pull and run on the VM

PERSON 3

DO — COMMANDS FILE B3 (IN THE VM SSH WINDOW)

```
docker image ls
cd ~/gridpulse
./deploy/vm_deploy.sh
```

1. Open the SSH window from VM instances → gridpulse-vm → **SSH**, then run the commands above. The deploy takes about a minute; talk over it.
2. Point at the backend digest it prints: **687b32e2...**, the same as the laptop.
3. Browser **<http://35.184.90.182>**: Dashboard → Regional forecast → **Recompute forecast**. Then **<http://35.184.90.182:8000/docs>**.

SAY

"Now we're on our GCP VM, gridpulse-vm. First, docker image ls, and there are no GridPulse images here. Now we run our deploy script. It does docker compose pull, then up with --no-build, so nothing is compiled on this VM: no pip install, no npm, no model training. You can see the layers downloading from Artifact Registry, and the database, backend and frontend all come up healthy. And here are the digests it pulled: the backend starts with 6-8-7-b-3-2-e-2, exactly the same as on the laptop and in the registry. So it really is byte-for-byte the same artifact. And here's the app at 35.184.90.182, the same URL we used in Milestone 2, and the forecast works here too."

8:15–9:00 · Budget alerts and current spend

PERSON 3

SHOW

1. Budgets & alerts → the GridPulse budget: **\$50** monthly, scope **gridpulse-509202**, thresholds **10, 20, 25, 50, 100%** (Actual), email alerts on.
2. Billing → Overview: this month's cost **\$2.42**, savings \$2.42, total **\$0.00**; \$289 of \$300 trial credit left.

SAY

"Our cost guardrails from Milestone 2 are still on. This is our monthly budget of 50 dollars, scoped only to the GridPulse project. It sends email alerts at 20, 50 and 100 percent, and we also added 10 and 25 percent for earlier warnings. It counts usage before our free-trial credits, so the alerts track what we actually use. This month we've used 2 dollars 42 so far, about 5 percent of the budget, all covered by trial credits. And to keep costs down, the registry and the VM are in the same region so pulls are free, and a cleanup policy deletes old untagged images after 7 days."

Hand-off: "[Person 2] will wrap up what changed since Milestone 2."

9:00–9:45 · What's easier than Milestone 2

PERSON 2

SHOW

Slide 2 (Before and after).

SAY

"So what's easier than Milestone 2? Last time we went through six manual stages on the VM: install Postgres and create the database, set up Python and pip install the ML packages, install Node and rebuild React with the VM's IP, copy the code and .env, seed the database and train the model, and then start everything by hand. Now it's two commands: build and push once from the laptop, and on the VM, git pull and one deploy script. The database is created from .env and seeds itself, and the model ships inside the image. And the specific problem it solved for us: our React build had the API address localhost:8000 baked in, so a build that worked on a laptop pointed to the wrong place on the VM. Now the frontend calls relative /api paths and nginx forwards them, so the exact same image works on localhost and on the VM."

9:45–10:00 · Close

PERSON 1 · ALL ON CAMERA

SAY

"That's GridPulse Milestone 3: one versioned image, built once and running identically on our laptop and on Google Cloud. And in the next milestones we'll take these same images onto managed services, so GridPulse can scale without anyone babysitting a VM. Thank you!"

Running long? Trim the Dockerfile segment first: the evidence page and one sentence about the password are enough.

After recording

- ☐ Watch it once: all three faces visible during their parts, audio clear, about 10 minutes.
- ☐ Upload to your UH OneDrive (CougarNet login) → Share with **sdesini@cougarnet.uh.edu**, **krajan@cougarnet.uh.edu**, **skolanu@cougarnet.uh.edu** (view or edit) → Copy link.
- ☐ Canvas: the OneDrive link, `gridpulse_milestone3.pptx`, the repo link (github.com/Balavardhanreddy5872/Gridpulse), `GridPulse_M3_Evidence_Screenshots.pdf`, `m3_vm_pull_and_run.log`, `GridPulse_M3_Written_Analysis.pdf`.